Flutter Clean Architecture

Universal API Integration Guide

by huthaifa Ma'in [linkedIn]

Dio · BLoC / Cubit · Clean Architecture

Introduction

This guide provides a repeatable, endpoint-agnostic blueprint for connecting any Flutter feature to any backend endpoint using **Dio for networking**, **Cubit for state management**, and **Clean Architecture for code organisation.**

Every time you receive a new API document from your backend team, **follow the same 10-step path described here**. The only things that change between features are the **endpoint URL**, the **field names**, and **the Dart class names** — **the structure never changes**.

How to Read an API Document

When you open a backend API document, these are the **five** things you must identify and note before writing a single line of code:

1. **The Method (GET / POST / PUT / DELETE)** — tells you how to communicate. POST sends data to create or trigger something. GET retrieves data without a body. PUT updates an existing resource. DELETE removes a resource.
2. **The Path (e.g. /api/products)** — the exact path you append to your base URL. Copy it character-for-character.
3. **The Request Body** — what the backend expects you to send. For GET and DELETE requests this is usually empty; for POST and PUT it is a JSON object. The keys you find here are the fields your UI must collect.
4. **The Response Body (Success)** — what the backend returns on success. The keys and nested objects here define exactly what your Entity and Model must contain.
5. **The Error Codes** — the HTTP status codes listed as failures (e.g. 400, 401, 404, 422). These are the exceptions your Repository must catch and convert into friendly UI messages.

💡 **TIP:** The response body is your single source of truth for Step 2 (Entity) and Step 3 (Model). Every field you need in your UI must be declared there.

The Data Flow Roadmap

Every feature you build follows the same path from the deepest configuration layer outward to the user's screen:

```
1. Endpoint URL
   ↓
2. Entity (Pure Dart blueprint)
   ↓
3. Model (JSON parser — extends Entity)
   ↓
4. Remote Data Source (Dio call)
   ↓
5. Repository Interface (domain contract)
   ↓
6. Repository Implementation (error handling)
   ↓
7. UseCase (single-action bridge)
   ↓
8. Cubit + States (state management)
   ↓
9. Dependency Injection (wiring with get_it)
   ↓
10. UI Screen (BlocProvider + BlocConsumer)
```

Start at Step 1 and drive outward. Never skip a layer, and never let a layer reach past its immediate neighbour.

Step 1 Add the Endpoint URL

Store every endpoint path in one central constants file. This ensures that if a path ever changes, you update it in a single place.

File: lib/core/api/end_points.dart

```
class EndPoint {
  // Platform-aware base URL
  static String get baseUrl {
    if (Platform.isAndroid) {
      return 'http://10.0.2.2:8000';  // Android emulator
    } else if (Platform.isIOS) {
      return 'http://localhost:8000';  // iOS simulator
    } else {
      return 'https://your-production-api.com';
    }
  }

  // ── Auth ──────────────────────────────────────────────
  static String login    = '/auth/login';
  static String register = '/auth/register';
  static String logout   = '/auth/logout';

  // ── Feature Example ───────────────────────────────────
  // static String featureName = '/api/path';
}
```

📌 **NOTE:** One line per endpoint. When your backend team sends you a new path, add it here first before touching any other file.

Step 2 Create the Entity

An Entity is a pure Dart class that holds only the fields your app actually uses. It contains zero JSON logic and zero framework dependencies — it belongs entirely to the domain layer.

Folder: lib/features/[feature]/domain/entities/

Template

```
// [Name]Entity – declare only the fields your UI needs.
// Source: the 'success' response body of your API doc.

class ProductEntity {
  final int    id;
  final String name;
  final double price;
  final String category;

  const ProductEntity({
    required this.id,
    required this.name,
    required this.price,
    required this.category,
  });
}
```

When the response contains a nested object

If the API returns a nested JSON object (e.g. a user that contains a company), create a separate Entity for the nested object and reference it as a typed field:

```
// company_entity.dart
class CompanyEntity {
  final int    id;
  final String name;
  const CompanyEntity({ required this.id, required this.name });
}

// user_entity.dart
import 'company_entity.dart';

class UserEntity {
  final String        token;
  final int           id;
  final String        name;
  final CompanyEntity company;  // <-- typed nested entity

  const UserEntity({
    required this.token,
    required this.id,
    required this.name,
    required this.company,
  });
}
```

💡 **TIP:** Look at your API's success response body. Every JSON key you need in your UI becomes a field here. Fields you will never display can be omitted.

Step 3	Create the Model

A Model extends its matching Entity and adds a fromJson factory constructor. It acts as a translator, converting the raw JSON map from the backend into a typed Dart object.

Folder: lib/features/[feature]/data/models/

Template

```
import '../../domain/entities/product_entity.dart';

class ProductModel extends ProductEntity {
  const ProductModel({
    required super.id,
    required super.name,
    required super.price,
    required super.category,
  });

  // Keys MUST exactly match the JSON keys in the API doc.
  factory ProductModel.fromJson(Map<String, dynamic> json) {
    return ProductModel(
      id:       json['id'],
      name:     json['name'],
      price:    (json['price'] as num).toDouble(),
      category: json['category'],
    );
  }
}
```

When the response contains a nested object

```
import '../../domain/entities/user_entity.dart';
import 'company_model.dart';

class UserModel extends UserEntity {
  const UserModel({
    required super.token,
    required super.id,
    required super.name,
    required super.company,
  });

  factory UserModel.fromJson(
    Map<String, dynamic> userJson,
    String               token,
    Map<String, dynamic> companyJson,
  ) {
    return UserModel(
      token:   token,
      id:      userJson['id'],
      name:    userJson['name'],
      company: CompanyModel.fromJson(companyJson),
    );
  }
}
```

When the response returns a list

```
// Inside ProductModel:
static List<ProductModel> fromJsonList(List<dynamic> jsonList) {
  return jsonList
    .map((item) => ProductModel.fromJson(item as Map<String, dynamic>))
    .toList();
}
```

💡 **TIP:** The JSON keys inside fromJson must be copied character-for-character from your API doc. A single typo causes a null value or a runtime crash.

Step 4	Write the Remote Data Source

The Remote Data Source is the only place in your app that talks to the network. It calls Dio, passes the correct request body, and returns a typed Model from the response.

Folder: lib/features/[feature]/data/datasources/

Template — POST (sends a body)

```
import '../../../../core/api/end_points.dart';
import '../../../../core/api/dio_client.dart';
import '../models/product_model.dart';

abstract class ProductRemoteDataSource {
  Future<ProductModel> createProduct(String name, double price);
}

class ProductRemoteDataSourceImpl implements ProductRemoteDataSource {
  final DioClient dioClient;
  ProductRemoteDataSourceImpl({ required this.dioClient });

  @override
  Future<ProductModel> createProduct(String name, double price) async {
    final response = await dioClient.post(
      EndPoint.createProduct,
      data: {
        'name':  name,   // keys must match the API doc request body
        'price': price,
      },
    );
    // Adjust the path below to match your API's response structure.
    final data = response.data['data'];
    return ProductModel.fromJson(data);
  }
}
```

Template — GET (retrieves a single object)

```
Future<ProductModel> getProduct(int id) async {
  final response = await dioClient.get(
    '${EndPoint.products}/$id',   // path parameter
  );
  return ProductModel.fromJson(response.data['data']);
}
```

Template — GET (retrieves a list)

```
Future<List<ProductModel>> getProducts() async {
  final response = await dioClient.get(EndPoint.products);
  // The API returns a JSON array - use fromJsonList.
  final List<dynamic> list = response.data['data'];
  return ProductModel.fromJsonList(list);
}
```

Template — DELETE (no meaningful response body)

```
Future<void> deleteProduct(int id) async {
  await dioClient.delete('${EndPoint.products}/$id');
  // Nothing to parse - the success case is simply no exception thrown.
}
```

📌 **NOTE:** Always dig into the correct key of response.data before passing it to fromJson. Check your API doc's success response to know the exact path (e.g. data['data'], data['result'], data['items']).

Step 5 — Write the Repository Interface

The Repository Interface is a contract that lives in the domain layer. It declares what actions are available without saying anything about how they are performed. This is what your UseCases and tests will depend on.

Folder: lib/features/[feature]/domain/repositories/

Template

```
import 'package:dartz/dartz.dart';
import '../../../../core/error/failures.dart';
import '../entities/product_entity.dart';

abstract class ProductRepository {
  // Return type pattern: Either<Failure, YourReturnType>
  Future<Either<Failure, ProductEntity>>       createProduct(String name, double price);
  Future<Either<Failure, ProductEntity>>       getProduct(int id);
  Future<Either<Failure, List<ProductEntity>>> getProducts();
  Future<Either<Failure, void>>                deleteProduct(int id);
}
```

🎯 **Rule:** Always use Either<Failure, T> from the dartz package. The Left side carries a Failure (error); the Right side carries your data (success). This forces safe error handling throughout your codebase.

Step 6	Implement the Repository

The Repository Implementation lives in the data layer and fulfils the contract defined in Step 5. It wraps every data source call in a try-catch block and converts exceptions into typed Failures.

Folder: lib/features/[feature]/data/repositories/

Template

```
import 'package:dartz/dartz.dart';
import '../../../../core/error/exceptions.dart';
import '../../../../core/error/failures.dart';
import '../../../../core/error/failures_mapper.dart';
import '../datasources/product_remote_data_source.dart';
import '../../domain/entities/product_entity.dart';
import '../../domain/repositories/product_repository.dart';

class ProductRepositoryImpl implements ProductRepository {
  final ProductRemoteDataSource remoteDataSource;
  ProductRepositoryImpl({ required this.remoteDataSource });

  @override
  Future<Either<Failure, ProductEntity>> createProduct(
    String name,
    double price,
  ) async {
    try {
      final model = await remoteDataSource.createProduct(name, price);
      return Right(model);              // ✅ success
    } on AppException catch (e) {
      return Left(mapExceptionToFailure(e));  // ❌ known API error
    } catch (e) {
      return Left(UnknownFailure(e.toString())); // ❌ unexpected crash
    }
  }

  // Repeat the same try-catch pattern for every method.
  @override
  Future<Either<Failure, List<ProductEntity>>> getProducts() async {
    try {
      final models = await remoteDataSource.getProducts();
      return Right(models);
    } on AppException catch (e) {
      return Left(mapExceptionToFailure(e));
    } catch (e) {
      return Left(UnknownFailure(e.toString()));
     }
   }
}
```

Step 7	Create the UseCase

A UseCase is a thin bridge class with a single call() method. The UI never talks directly to the Repository — it always goes through a UseCase. This keeps business logic out of your widgets and makes testing straightforward.

Folder: lib/features/[feature]/domain/usecases/

Template — with parameters

```
import 'package:dartz/dartz.dart';
import '../../../../core/error/failures.dart';
import '../entities/product_entity.dart';
import '../repositories/product_repository.dart';

class CreateProductUseCase {
  final ProductRepository repository;
  CreateProductUseCase({ required this.repository });

  Future<Either<Failure, ProductEntity>> call(
    String name,
    double price,
  ) async {
    return await repository.createProduct(name, price);
  }
}
```

Template — no parameters (e.g. get all items)

```
class GetProductsUseCase {
  final ProductRepository repository;
  GetProductsUseCase({ required this.repository });

  Future<Either<Failure, List<ProductEntity>>> call() async {
    return await repository.getProducts();
  }
}
```

📌 **NOTE:** Always name the method 'call'. This lets you invoke the UseCase instance directly — e.g. await getProductsUseCase() — which reads cleanly inside the Cubit.

Step 8	Create the States and Cubit

The Cubit manages UI state. It calls the UseCase, then emits one of four possible states based on the result. The states file and the cubit file are kept in the same folder.

Folder: lib/features/[feature]/presentation/manager/

States template — [feature]_state.dart

```
import '../../domain/entities/product_entity.dart';

abstract class ProductState {}

// 1. App has not triggered any action yet.
class ProductInitial extends ProductState {}

// 2. API call is in progress — show a spinner.
class ProductLoading extends ProductState {}

// 3. API call succeeded — carry the data the UI needs.
class ProductSuccess extends ProductState {
  final ProductEntity product;
  ProductSuccess({ required this.product });
}

// 3b. For list endpoints, carry a list instead:
class ProductListSuccess extends ProductState {
  final List<ProductEntity> products;
  ProductListSuccess({ required this.products });
}

// 4. API call failed — carry the friendly error message.
class ProductError extends ProductState {
  final String message;
  ProductError({ required this.message });
}
```

Cubit template — [feature]_cubit.dart

```
import 'package:flutter_bloc/flutter_bloc.dart';
import '../../domain/usecases/create_product_usecase.dart';
import '../../domain/usecases/get_products_usecase.dart';
import 'product_state.dart';

class ProductCubit extends Cubit<ProductState> {
  final CreateProductUseCase createProductUseCase;
  final GetProductsUseCase   getProductsUseCase;

  ProductCubit({
    required this.createProductUseCase,
    required this.getProductsUseCase,
  }) : super(ProductInitial());

  Future<void> createProduct(String name, double price) async {
    emit(ProductLoading());
    final result = await createProductUseCase(name, price);
    result.fold(
      (failure) => emit(ProductError(message: failure.message)),
      (product) => emit(ProductSuccess(product: product)),
    );
  }

  Future<void> getProducts() async {
    emit(ProductLoading());
    final result = await getProductsUseCase();
    result.fold(
      (failure) => emit(ProductError(message: failure.message)),
      (products) => emit(ProductListSuccess(products: products)),
    );
  }
}
```

🔑 **Key Rule:** Use result.fold() to unpack the Either. The first callback receives a Failure (Left) and the second receives your data (Right). Never access result.getOrElse or throw — always fold.

Step 9	Dependency Injection

Use get_it as a service locator to wire all classes together. You register each class once and then retrieve it anywhere in the app using sl<ClassName>().

File: lib/core/di/service_locator.dart

Registration order — always follow this sequence

```
// 1. Core infrastructure (register once, at the top)
sl.registerLazySingleton<Dio>(() => Dio());
sl.registerLazySingleton<CacheHelper>(() => CacheHelper());
sl.registerLazySingleton<DioClient>(() => DioClient(cacheHelper: sl()));

// 2. Data Sources  (data layer)
sl.registerLazySingleton<ProductRemoteDataSource>(
  () => ProductRemoteDataSourceImpl(dioClient: sl()),
);

// 3. Repositories  (data layer implements domain contract)
sl.registerLazySingleton<ProductRepository>(
  () => ProductRepositoryImpl(remoteDataSource: sl()),
);

// 4. UseCases  (domain layer)
sl.registerLazySingleton(() => CreateProductUseCase(repository: sl()));
sl.registerLazySingleton(() => GetProductsUseCase(repository: sl()));

// 5. Cubits  - ALWAYS use registerFactory, not registerLazySingleton.
//    A Factory creates a fresh Cubit instance each time the screen opens,
//    so state never leaks between navigation sessions.
sl.registerFactory(() => ProductCubit(
  createProductUseCase: sl(),
  getProductsUseCase:   sl(),
));
```

⚠️ **Important:** Never register a Cubit as registerLazySingleton. Always use registerFactory. A singleton Cubit persists its state across all screens that use it, causing stale data bugs that are very hard to trace.

Step 10	Connect the UI

Wrap your screen in a BlocProvider to inject the Cubit, and use BlocConsumer to listen for one-time side effects (navigation, snackbars) and rebuild the UI when state changes.

Folder: lib/features/[feature]/presentation/views/

Template

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import '../../manager/product_cubit.dart';
import '../../manager/product_state.dart';
import '../../../../../core/di/service_locator.dart';

class ProductScreen extends StatelessWidget {
  const ProductScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return BlocProvider(
      create: (_) => sl<ProductCubit>()..getProducts(), // inject + auto-load
      child: BlocConsumer<ProductCubit, ProductState>(

        // LISTENER: runs once per state change — use for navigation & snackbars.
        listener: (context, state) {
          if (state is ProductError) {
            ScaffoldMessenger.of(context).showSnackBar(
              SnackBar(content: Text(state.message), backgroundColor: Colors.red),
            );
          }
          if (state is ProductSuccess) {
            // Navigate, show confirmation, etc.
          }
        },

        // BUILDER: runs every time state changes — returns a Widget.
        builder: (context, state) {
          if (state is ProductLoading) {
            return const Center(child: CircularProgressIndicator());
          }
          if (state is ProductListSuccess) {
            return ListView.builder(
              itemCount: state.products.length,
              itemBuilder: (_, i) => ListTile(title: Text(state.products[i].name)),
            );
          }
          // ProductInitial or ProductError — show default/empty screen.
          return const SizedBox.shrink();
        },
      ),
    );
  }}
```

🎯 **Rule:** Listener is for side effects (things that happen once). Builder is for drawing widgets (things that rebuild on every state). Never put navigation inside the builder.

Variant Endpoint Patterns

The **10-step** blueprint handles the most common case — a POST that returns a single object. Below are the adjustments for the two other common patterns you will encounter.

Pattern A — GET endpoint that returns a list

The only steps that differ from the main blueprint are Step 3, Step 4, Step 5, Step 8, and the state type.

- Step 3 (Model): add the static `fromJsonList` helper shown in Step 3 above.
- Step 4 (Data Source): use `dioClient.get()` with no data body. Parse `response.data['data']` as `List<dynamic>` and call `fromJsonList`.
- Step 5 (Repository Interface): the return type becomes `Either<Failure, List<YourEntity>>`.
- Step 8 (States): add a separate Success state that carries a `List<YourEntity>` instead of a single entity.
- Step 10 (UI): check for the list success state in the builder and render a `ListView`.

Pattern B — DELETE / action endpoint with no meaningful response body

Some endpoints (DELETE, logout, mark-as-read, etc.) return only a status code with no data. The adjustments are:

- Step 2 (Entity): skip entirely — there is no data to model.
- Step 3 (Model): skip entirely.
- Step 4 (Data Source): call `dioClient.delete()` and return `void`. Success means no exception was thrown.
- Step 5 (Repository Interface): use `Either<Failure, void>` as the return type.
- Step 6 (Repository Impl): return `Right(null)` on success.
- Step 7 (UseCase): return type is `Either<Failure, void>`.

- Step 8 (States): your Success state carries no data — `class DeleteSuccess extends YourState {}`

Quick Reference — Checklist per Endpoint

When you receive a new API document, work through this checklist in order. Check each item off before moving to the next.

Step	What to produce
Step 1: Endpoint URL	Add path string to end_points.dart
Step 2: Entity	Pure Dart class — fields from API response body
Step 3: Model	Extends Entity, adds fromJson (+ fromJsonList for arrays)
Step 4: Remote Data Source	Dio call (GET / POST / PUT / DELETE) + JSON parsing
Step 5: Repository Interface	Abstract contract with Either<Failure, T> return types
Step 6: Repository Implementation	try/catch wrapping, Right() or Left(Failure)
Step 7: UseCase	Thin bridge with a call() method
Step 8: States + Cubit	4 states: Initial, Loading, Success, Error
Step 9: Dependency Injection	Register Data Source, Repository, UseCase, Cubit (Factory)
Step 10: UI Screen	BlocProvider + BlocConsumer (listener + builder)

Folder Structure Reference

Every feature follows the same directory layout inside lib/features/[feature_name]/:

```
lib/
├── core/
│   ├── api/
│   │   ├── end_points.dart        ← Step 1
│   │   └── dio_client.dart
│   ├── di/
│   │   └── service_locator.dart    ← Step 9
│   └── error/
│       ├── exceptions.dart
│       ├── failures.dart
│       └── failures_mapper.dart
│
└── features/
    └── [feature_name]/
        ├── domain/              ← Pure Dart, no Flutter/Dio dependencies
        │   ├── entities/        ← Step 2
        │   ├── repositories/      ← Step 5 (interface)
        │   └── usecases/         ← Step 7
        │
        ├── data/                ← Implements domain contracts
        │   ├── models/          ← Step 3
        │   ├── datasources/       ← Step 4
        │   └── repositories/      ← Step 6 (implementation)
        │
        └── presentation/         ← Flutter UI
            ├── manager/         ← Step 8 (cubit + states)
            └── views/           ← Step 10
```

Golden Rules

- **One UseCase per action.** GetProducts and CreateProduct are two separate UseCase classes, not one class with two methods.
- **Domain never imports from data or presentation.** The entities/ and repositories/ (interface) folders must never contain an import of Dio, Cubit, or any model.
- **Register Cubits with registerFactory.** All other classes can use registerLazySingleton.
- **fromJson keys must match the API doc exactly.** A wrong key silently produces null instead of a crash — always double-check.
- **fold() every Either.** Never access .getOrElse on an untested Either. Always fold and handle both sides.
- **Listener is for side effects; builder is for widgets.** Never navigate inside the builder.